Under the Hood

A complete, plain-language guide to the code, the project, and the algorithms

(detailed enough for a developer, readable enough for a curious beginner)

Chladni Plate Inverse-Design System — Department of Physics, Imperial College London

Contents

---

1. What the whole thing does (in one breath)

You draw a target picture. The software tries to design a square plate — by choosing how thick it is at each point on a 15×15 grid — and a vibration frequency, so that when the real plate is shaken at its centre and sprinkled with sand, the sand settles into a pattern that looks like your target. Crucially, it also tells you, *before* wasting any effort, whether your picture is even physically possible.

The data flows in one direction, through a chain of clearly separated steps:

```
your drawing → clean target + "is this possible?"  verdict → fast surrogate
search (thickness + frequency) → accurate COMSOL simulation (pick the best
vibration) → trust-region fine-tuning → score against "sand physics" → printable
STL + report
```

Almost every folder in `src/` is one link in that chain. The rest of this guide walks the chain link by link: first the *map* of the code (Section 2), then each *algorithm* in its own section, then a full *end-to-end* walkthrough (Section 12) and how to *run* it (Section 13).

2. The map of the code

The project is a Python package (`src/`) plus a web front-end (`frontend/`), a set of COMSOL/-MATLAB templates (`comsol_templates/`), and data/report folders. Here is what each part is for.

2.1 Entry points and configuration

- `src/main.py` — the command-line front door. Sub-commands include `target-ui` (start the web app), `prepare-target`, `generate-candidates`, `run-workflow`, `diagnose-comsol`, `self-test`.
- `config.yaml` + `src/config.py` — one central settings file (plate size, grid size, material numbers, scoring weights, COMSOL/MATLAB paths) loaded once and handed to every module as a `config` dictionary. If you want to change the plate or the material, you change it here, not in the algorithms.

2.2 The application: front-end + backend bridge

- `frontend/target_designer.html` — a single-page app with tabs **Target / Tune / Results / Run / Gallery / Export**. It is just buttons, a drawing canvas, and JavaScript; it never touches COMSOL directly.
- `src/frontend/target_ui_server.py` — the Python web server (∼2,500 lines) that sits behind the page. It runs the pipeline, manages the COMSOL connection, streams progress, can be cancelled mid-run with a Stop button, and serves results. Think of it as the bridge between the HTML and the algorithms.

2.3 The algorithm core (the focus of this guide)

- `src/target/` — turn a drawing into a clean black-and-white target and judge whether it is realisable (Section 3).
- `src/symmetry/` — the symmetry "gatekeeper": the D_4 decomposition that produces the all-important reachability number (Section 4).
- `src/physics/` — the fast, differentiable plate "sketch" and its orthotropic (directional-material) cousin (Sections 5–6).
- `src/scoring/` — how a pattern is graded, from old pixel-overlap metrics to the powder-physics "recognisability" score (Section 7).
- `src/optimisation/` — the actual search engines, from random search up to the W10 gradient optimiser and the two-phase production pipeline (Sections 8–9).
- `src/comsol/` — everything that talks to the real COMSOL simulator through MATLAB LiveLink (Section 10).
- `src/export/` — turn a finished thickness field into a printable STL (Section 11).

2.4 Supporting machinery

- `src/forced_response/`, `src/frequency_domain/`, `src/nodal/` — drive the plate at a frequency, read the response, and extract nodal lines from a simulated field.
- `src/subspace/` — the data-driven design spaces (PCA "manifold" of past candidates, mode-shape bases) used by the smarter searches.
- `src/uniform_pattern/` — a catalogue of plain-plate vibration modes you can browse and gently perturb, filtered by whether a central drive can actually excite them.
- `src/topology_grammar/`, `src/tags/`, `src/candidate/` — generate and constrain thickness candidates (keep them inside the 0.6–2.0 mm range and the neighbour-difference limit).
- `src/visualisation/` — make the preview images you see in the UI.
- `comsol_templates/` — the parameterised COMSOL model (`.mph`) and the MATLAB script LiveLink runs for each candidate.

3. Algorithm 1 — From a drawing to an honest verdict

Folders: `src/target/`

> **In plain words:** before trying to build anything, a good engineer asks "is this even possible?". This first step cleans up your sketch and then gives it a blunt health-check: *easy*, *needs a topology change*, or *forget it*.

What actually happens. `preprocess_target.py` converts whatever you draw or upload into a clean binary image (foreground = where you want a nodal line). `analyse_target.py` measures basic health: how much of the canvas is filled, how many separate pieces it has, how thin the strokes are.

Then `target_realizability.py` issues a **verdict** by combining several physically-motivated red flags:

- a **Courant index** — roughly, "how high up the list of vibration modes would you have to go to draw this many separate regions?" (based on the classic nodal-domain counting theorem). Too high = unrealistic.
- a **stroke-gap** check — nodal lines cannot be packed closer than the physics allows; thin, tightly-spaced strokes are flagged.
- a D_4 **symmetry mismatch** — how far the picture is from the plate's four-fold symmetry (this is the cheap preview of the deep result in Section 4).
- counts of connected pieces, interior holes, and skeleton endpoints.

The thresholds (e.g. "infeasible if Courant index > 60", "needs topology if D_4 mismatch > 0.55") live in one dictionary at the top of the file, so the verdict is transparent and tweakable.

Why it matters. The single biggest lesson of the whole project was that this verdict should be used *up front*. The letters "IC" were doomed from the start, and this module could say so in milliseconds — long before any expensive simulation.

4. Algorithm 2 — The symmetry gatekeeper (D_4 and the reachability number)

File: `src/symmetry/d4_decomposition.py` (about 100 lines, and the mathematical heart of the project)

> **In plain words:** a square shaken from its exact centre can only ever make patterns that have the same symmetry as the square. This module measures what fraction of your target has that symmetry — and that fraction is a hard ceiling on how well you can ever do.

What actually happens. The square's symmetry group D_4 has eight moves: four rotations ($0°, 90°, 180°, 270°$) and four mirror flips (horizontal, vertical, two diagonals). The code implements each move as a simple array operation (`np.rot90`, `np.flipud`, transpose, ...).

Any picture P can be split into five "symmetry flavours" (the **irreducible representations** A_1, A_2, B_1, B_2, E) using a standard projection formula. For each flavour ρ the code computes

$$P_\rho \;=\; \frac{\dim \rho}{8} \sum_{g \in D_4} \chi_\rho(g)\,(g \cdot P),$$

i.e. apply all eight moves, weight each by a ± 1 (or $0, 2$) number from the **character table**, and add them up. The five pieces are perfectly orthogonal and add back to the original (the file even ships a `check_orthogonality` sanity test).

The punchline is one function, `coupling_accessibility`: a central point drive can only excite the fully-symmetric flavour A_1, so

$$\text{reachability}(P) \;=\; \frac{\|P_{A_1}\|^2}{\|P\|^2}.$$

For an X-shape this is 100%; for a single diagonal about 52%; for "IC" about 42%. The rest of the picture's energy lives in flavours the centre drive literally cannot touch.

Why it matters. This converts "our algorithm isn't good enough yet" into a precise, provable "58% of IC is physically forbidden." It is the mathematical backbone behind the verdict of Section 3 and the whole project's honesty.

5. Algorithm 3 — The fast physics "sketch" (the differentiable surrogate)

File: `src/physics/differentiable_plate.py`

> **In plain words:** to search through millions of plate designs we need a *fast* way to guess how each one vibrates. This is a stripped-down physics model that runs in a fraction of a second — and, importantly, it is built so a computer can automatically work out "which way should I nudge the thickness to make this better?"

What actually happens. The plate is put on a grid (an odd size like 25×25 so there is a true centre cell). The model builds two ingredients:

- a **stiffness matrix** K and a **mass** M. Stiffness comes from the thin-plate bending law: each cell's bending stiffness is $D = \frac{E\,h^3}{12(1-\nu^2)}$ (thicker $\Rightarrow$ much stiffer, because of the h^3), and $K = L^{\mathsf{T}}\,\mathrm{diag}(D)\,L$ where L is a simple finite-difference Laplacian. Mass is just "how much material sits at each cell".

- the **forced response**. Shaking the centre at angular frequency ω and solving the linear system $(K + \mathrm{i}\,\omega C - \omega^2 M)\,u = f$ gives the complex vibration amplitude u at every point. Where $|u|$ is small, sand will collect — that is the predicted Chladni pattern.

The clamp at the centre is imposed by simply removing those grid points from the solve.

The whole thing is written in **PyTorch**, which means it is **differentiable**: PyTorch can automatically compute the gradient of any score with respect to all 225 thickness values at once (**automatic differentiation**). That gradient is what makes a smart, directed search possible instead of blind guessing.

Why it matters — and the catch. This model is a fast *compass*, not a *judge*. It is systematically over-optimistic (it treats the plate as paper-thin and ignores 3-D effects), so it can promise an 8–18× improvement that the real simulator only partly delivers. The design rule of the whole project follows from this: *use the surrogate to choose a direction, never to declare victory.*

6. Algorithm 4 — The material twist, and why we froze it

File: `src/physics/orthotropic_plate.py`; used by W10

> **In plain words:** 3D-printed plastic is slightly stronger along its print lines. We built a version of the plate model where each cell's "grain direction" θ could be rotated, hoping it would let us cheat the symmetry. It didn't help, so we froze it.

What actually happens. `orthotropic_plate.py` assembles the same kind of stiffness matrix, but from a directional (**orthotropic**) bending law (from Reddy's laminated-plate theory): each cell can rotate its stiffness tensor by an angle θ. In principle, varying θ per cell breaks the perfect symmetry and could let a central drive reach the "forbidden" flavours of Section 4.

Why it was frozen. In the frequency range the optimiser actually uses, the mass term $\omega^2 M$ dwarfs the stiffness term K by about five orders of magnitude — and θ only enters through K. So its effect on the answer is buried under numerical noise (the measured sensitivity is a few parts per million). Worse, θ adds a wobbly extra dimension that the optimiser wastes effort on. A 10-run experiment (5 targets × 2 materials) confirmed that ordinary isotropic PLA beats "optimised" carbon-fibre PETG on most targets. So in `recognisability_placement_w10.py` the orientation field is held at zero (the material is still orthotropic, every cell just points the same way), and the optimiser only moves thickness H and frequency ω.

7. Algorithm 5 — Grading like real sand (the recognisability score)

File: `src/scoring/recognisability_score.py`

> **In plain words:** how do we score "does this look like the target?" Early on we compared pixels, which turned out to be the wrong ruler. The fix was to score the way *sand* actually behaves.

What actually happens. Sand piles up where the vibration amplitude is near zero. The function `chladni_powder_density` turns an amplitude field into a predicted sand map with a Gaussian, powder $= \exp(-(|u|/\sigma)^2)$, after dividing $|u|$ by its *maximum* (a deliberate choice — an earlier version added a tiny constant, which silently destroyed all the COMSOL results because COMSOL amplitudes are around 10^{-12}). From the sand map it computes four numbers:

- **enrichment** — how much denser the sand is on the target than on average (the headline number; "3.76×" means 3.76 times denser).
- **recall** — what fraction of the target line is actually covered by low-amplitude valley.
- **contrast** — target-region sand density versus background.
- **direction** — whether the sand pattern points the same way as the target (a cosine between their principal axes).

These combine into one **composite** score with fixed weights (0.40 enrichment + 0.30 recall + 0.20 contrast + 0.10 direction).

Why it matters. Switching from pixel overlap (`src/scoring/metrics.py`, IoU/Dice) to this powder model re-ranked the entire candidate history — some of the best designs had been hiding under the wrong metric for months.

8. Algorithm 6 — The search engines

Folder: `src/optimisation/`

> **In plain words:** given a target and a way to score, how do we actually *find* a good thickness map? The project tried about twenty methods; here are the ones that survived.

The folder is a museum of the three eras (blind search → structured → recognisability) plus the modern production code:

- `random_search.py` — the Era-I baseline: generate many random thickness maps and keep the best. Robust, but it only ever finds generic blobs.
- `gradient_optimizer.py` — the key upgrade: because the surrogate is differentiable, we can follow the gradient "downhill". It also handles the **reparametrisation** trick that keeps thickness inside 0.6–2.0 mm (a sigmoid maps unbounded numbers into the legal range) and a neighbour-smoothness penalty so the plate stays printable.
- `homotopy.py` — start from a pattern the plate *wants* to make and slowly "morph" the goal toward your target; where it stalls is an honest measure of reachability.
- `spectral_placement.py` — push several of the plate's natural frequencies to cluster near the drive frequency.
- `multifreq_placement.py` — drive at several frequencies at once (later found to be a partly-illusory trick).
- `recognisability_placement_w10.py` — **the modern optimiser**. It jointly tunes thickness H and the drive frequencies ω using the **Adam** gradient method (∼250 steps) on the orthotropic surrogate, scoring with the powder-physics loss. This is the "compass" stage of the production pipeline.
- `trust_region_w9.py` — the first attempt at a surrogate↔COMSOL correction loop; structurally right but under-tuned. Its ideas were folded into Phase 2 of the pipeline below.

9. Algorithm 7 — The two-phase production pipeline (the customer path)

File: `src/optimisation/production_pipeline.py` (∼1,100 lines)

In plain words: this is the big red "run it for real" button. It chains the fast search and the accurate simulator together in the right order, so that the quick model only ever suggests, and the trustworthy simulator decides.

What actually happens. The file documents its own eight stages:

S1. Prepare the target (from PNG/NPY).

S2. W10 surrogate search — get a good thickness H and candidate frequencies fast (Section 8).

S3. COMSOL eigen-analysis — ask the real simulator for the plate's ∼30 true natural modes.

S4. Phase 1 — modal selection — rank those modes by how much they look like the target, and also try a few clever "off-resonance beat" frequencies.

S5. COMSOL forced response at the selected frequencies (the real vibration, not the sketch).

S6. Composite search — exhaustively try combining up to three modes (RMS / MAX / SUM) and keep the combination that scores best.

S7. Phase 2 — trust-region refinement (optional) — take the real simulator as ground truth and make small, safe tweaks to H: the surrogate proposes a step, COMSOL re-checks it, and the step is accepted or the "trust radius" shrinks. Modes are matched between iterations with the **MAC** (modal assurance criterion).

S8. Package the deliverables — `H.csv`, a summary JSON, the composite sand image, density maps, and visualisations.

Why this shape. It cleanly separates the two things that can go wrong: *which frequency* (fixed by driving at COMSOL's true resonance) and *which shape* (fixed by the small trust-region tweaks). A naive loop that corrects both at once tends to diverge — which is exactly what the earlier `trust_region_w9.py` taught us. A cooperative `Stop` hook lets the UI kill a long COMSOL child process promptly.

10. Algorithm 8 — Talking to the real simulator (the COMSOL bridge)

Folder: `src/comsol/ + comsol_templates/`

In plain words: the trustworthy physics lives in COMSOL, a professional simulator driven through MATLAB. This folder is the diplomatic corps that gets Python and COMSOL to talk to each other reliably on any machine.

What actually happens. `discovery.py` finds where COMSOL and MATLAB are installed (or already running); `diagnostics.py` checks the machine is ready; `server.py` starts or connects to a COMSOL `mphserver`; `run_livelink.py` hands a candidate's parameters to the MATLAB script `run_chladni_candidate.m`, which loads the parameterised model `Chladni_15x15_parameterized.mph`, runs it, and exports frequencies and mode shapes; `import_results.py` reads those exports back into NumPy.

Why it matters. This is the most deployment-sensitive part of the project (it depends on licences, paths, and the exact `.mph` model). It is deliberately isolated so the algorithms above never need to know the messy details of how the simulation actually runs.

11. Algorithm 9 — From a design to a thing you can hold (STL export)

File: `src/export/stl_export.py`

> **In plain words:** the final thickness map is just 225 numbers. This turns them into a 3-D model you can send straight to a printer.

What actually happens. Each of the 15×15 cells becomes a little box, 10 mm × 10 mm wide and as tall as that cell's thickness, all sitting on a flat bottom at $z = 0$ (so it sticks to the print bed). The 225 boxes are written out as one binary STL "triangle soup" (2,700 triangles, ∼132 KB), with an optional Gaussian smoothing of the steps. The module deliberately uses *no* heavy 3-D libraries (just NumPy) so it runs even on a nearly-full disk.

12. Putting it together: one run, start to finish

Suppose you draw an X and hit **Run**:

1. The front-end (`frontend/target_designer.html`) sends your drawing to the backend (`src/frontend/target_ui_server.py`).
2. `src/target/` cleans it and `src/symmetry/` reports "reachability ≈ 100%" — green light.
3. The production pipeline (`src/optimisation/production_pipeline.py`) runs the W10 surrogate search (`src/physics/` + `src/scoring/`) to get a thickness map and candidate frequencies in seconds.
4. It asks COMSOL (`src/comsol/`) for the true modes, picks the ones that most look like an X, and drives them for real.
5. It tries combinations, optionally fine-tunes the thickness with the trust-region loop, and scores everything with the powder-physics metric.
6. It writes a report and, via `src/export/stl_export.py`, a printable `.stl`. The **Results** and **Export** tabs show you the sand image and let you download the plate.

For an "IC" target, step 2 would instead warn you that most of the picture is physically out of reach — saving you the entire rest of the run.

13. How to run it yourself

Everything goes through `src/main.py`:

```
python -m src.main self-test                                  # check the install
python -m src.main diagnose-comsol              # check COMSOL/MATLAB are reachable
python -m src.main target-ui                  # launch the web app (the usual way in)
```

The web app is the friendly path: the **Target** tab to draw/upload, **Tune** to set material, **Run** to launch the pipeline (with a live progress bar and a Stop button), **Results** to inspect, **Gallery** to browse plain-plate modes filtered by what a central drive can excite, and **Export** to download the printable plate. Power users can also call `prepare-target`, `generate-candidates`, and `run-workflow` directly.

14. Mini-glossary

Nodal line
A line on the plate that barely moves; sand collects here. The visible Chladni pattern is the set of nodal lines.

Surrogate
The fast, simplified physics model (`src/physics/`). A compass for the search, not the final judge.

COMSOL
The slow, accurate professional simulator; the ground-truth judge.

D_4 / irreducible representation
The square's symmetry group and its five "symmetry flavours". A central drive only reaches the fully-symmetric one (A_1).

Reachability
The fraction of your target that lives in A_1; a hard ceiling on how well you can do.

Enrichment
How many times denser the sand is on your target line than on average; the project's headline success number.

Trust region
A safe-step method: only make a change as big as you currently "trust", and shrink that trust if the change disappoints.

Adam
A popular gradient-based optimiser used by the W10 search.

STL
The 3-D file format a printer understands.

This guide is the plain-language companion to the full technical paper and the popular-science one-pager. Together they cover the same project at three depths: the deep end (paper), the shallow end (popular), and the engine room (this guide).